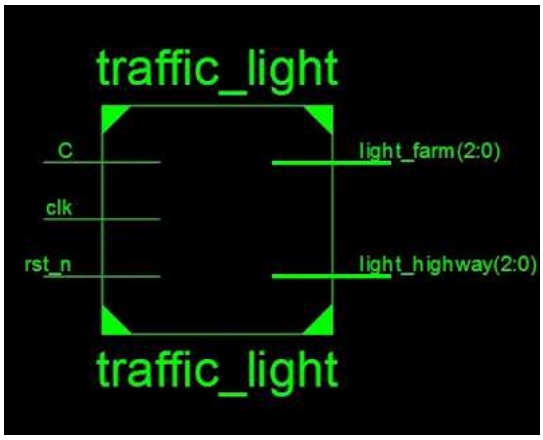
Verilog code for Traffic light controller

A Verilog source code for a traffic light controller on FPGA is presented. A sensor on the farm is to detect if there are any vehicles and change the traffic light to allow the vehicles to cross the highway. Otherwise, highway light is always green since it has higher priority than the farm.



Verilog code for traffic light controller:

```
// fpga4student.com FPGA projects, VHDL projects, Verilog projects
// Verilog project: Verilog code for traffic light controller
module traffic_light(light_highway, light_farm, C, clk, rst_n);
parameter HGRE_FRED=2'b00, // Highway green and farm red
   HYEL_FRED = 2'b01,// Highway yellow and farm red
   HRED_FGRE=2'b10,// Highway red and farm green
   HRED_FYEL=2'b11;// Highway red and farm yellow
input C, // sensor
   clk, // clock = 50 MHz
   rst_n; // reset active low
output reg[2:0] light_highway, light_farm; // output of lights
// fpga4student.com FPGA projects, VHDL projects, Verilog projects
reg[27:0] count=0,count_delay=0;
reg delay10s=0, delay3s1=0,delay3s2=0,RED_count_en=0,YELLOW_count_en1=0,YELLOW_cou
wire clk_enable; // clock enable signal for 1s
reg[1:0] state, next_state;
// next state
always @(posedge clk or negedge rst_n)
begin
if(~rst_n)
 state <= 2'b00;
else
 state <= next_state;
end
// FSM
always @(*)
begin
case(state)
HGRE_FRED: begin // Green on highway and red on farm way
 RED_count_en=0;
 YELLOW_count_en1=0;
 YELLOW_count_en2=0;
 light_highway = 3'b001;
 light_farm = 3'b100;
 if(C) next_state = HYEL_FRED;
 // if sensor detects vehicles on farm road,
 // turn highway to yellow -> green
```

```
 else next_state =HGRE_FRED;
end
HYEL_FRED: begin// yellow on highway and red on farm way
  light_highway = 3'b010;
  light_farm = 3'b100;
  RED_count_en=0;
 YELLOW_count_en1=1;
 YELLOW_count_en2=0;
  if(delay3s1) next_state = HRED_FGRE;
  // yellow for 3s, then red
  else next_state = HYEL_FRED;
end
HRED_FGRE: begin// red on highway and green on farm way
 light_highway = 3'b100;
 light_farm = 3'b001;
 RED_count_en=1;
 YELLOW_count_en1=0;
 YELLOW_count_en2=0;
 if(delay10s) next_state = HRED_FYEL;
 // red in 10s then turn to yello -> green again for high way
 else next_state =HRED_FGRE;
end
HRED_FYEL:begin// red on highway and yellow on farm way
 light_highway = 3'b100;
 light_farm = 3'b010;
 RED_count_en=0;
 YELLOW_count_en1=0;
 YELLOW_count_en2=1;
 if(delay3s2) next_state = HGRE_FRED;
 // turn green for highway, red for farm road
 else next_state =HRED_FYEL;
end
default: next_state = HGRE_FRED;
endcase
end
// fpga4student.com FPGA projects, VHDL projects, Verilog projects
// create red and yellow delay counts
always @(posedge clk)
begin
if(clk_enable==1) begin
 if(RED_count_en||YELLOW_count_en1||YELLOW_count_en2)
  count_delay <=count_delay + 1;
  if((count_delay == 9)&&RED_count_en)
  begin
   delay10s=1;
   delay3s1=0;
   delay3s2=0;
   count_delay<=0;
  end
  else if((count_delay == 2)&&YELLOW_count_en1)
  begin
   delay10s=0;
   delay3s1=1;
   delay3s2=0;
   count_delay<=0;
  end
  else if((count_delay == 2)&&YELLOW_count_en2)
  begin
   delay10s=0;
   delay3s1=0;
   delay3s2=1;
   count_delay<=0;
  end
```

```
  else
  begin
   delay10s=0;
   delay3s1=0;
   delay3s2=0;
  end
 end
end
// create 1s clock enable
always @(posedge clk)
begin
 count <=count + 1;
 //if(count == 50000000) // 50,000,000 for 50 MHz clock running on real FPGA
 if(count == 3) // for testbench
  count <= 0;
end
 assign clk_enable = count==3 ? 1: 0; // 50,000,000 for 50MHz running on FPGA
endmodule
```

Testbench Verilog code for functional simulation

```
// fpga4student.com FPGA projects, VHDL projects, Verilog project
// Verilog project: Verilog code for traffic light controller
`timescale 10 ns/ 1 ps
// 2. Preprocessor Directives
`define DELAY 1
// 3. Include Statements
//`include "counter_define.h"
module tb_traffic;
// 4. Parameter definitions
parameter ENDTIME  = 400000;
// 5. DUT Input regs
//integer count, count1, a;
reg clk;
reg rst_n;
reg sensor;
wire [2:0] light_farm;
// 6. DUT Output wires
wire [2:0] light_highway;

// fpga4student.com FPGA projects, VHDL projects, Verilog projects
// 7. DUT Instantiation
traffic_light tb(light_highway, light_farm, sensor, clk, rst_n);

// 8. Initial Conditions
initial
 begin
 clk = 1'b0;
 rst_n = 1'b0;
 sensor = 1'b0;
 // count = 0;
//// count1=0;
// a=0;
 end
// 9. Generating Test Vectors
initial
 begin
 main;
 end
task main;
 fork
 clock_gen;
 reset_gen;
 operation_flow;
```

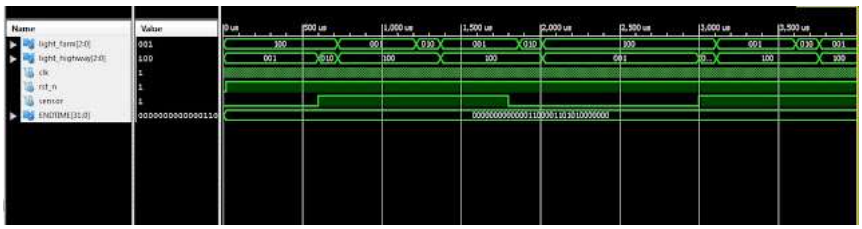
```
 debug_output;
 endsimulation;
 join
endtask
task clock_gen;
 begin
 forever #`DELAY clk = !clk;
 end
endtask

task reset_gen;
 begin
 rst_n = 0;
 # 20
 rst_n = 1;
 end
endtask

// fpga4student.com FPGA projects, VHDL projects, Verilog projects
task operation_flow;
 begin
 sensor = 0;
 # 600
 sensor = 1;
 # 1200
 sensor = 0;
 # 1200
 sensor = 1;
 end
endtask
// 10. Debug output
task debug_output;
 begin
 $display("----------------------------------------------");
        $display("------------------     -----------------------");
 $display("----------- SIMULATION RESULT ----------------");
 $display("--------------             -------------------");
 $display("----------------         ---------------------");
 $display("----------------------------------------------");
 $monitor("TIME = %d, reset = %b, sensor = %b, light of highway = %h, light of far
 end
endtask

// fpga4student.com FPGA projects, VHDL projects, Verilog projects
//12. Determines the simulation limit
task endsimulation;
 begin
 #ENDTIME
 $display("-------------- THE SIMUALTION END ------------");
 $finish;
 end
endtask

endmodule
```

Simulation results for the traffic light controller in Verilog:

```
TIME = 2000, reset = 1, sensor = 0, light of highway = 1, light of farm road = 4
TIME = 60000, reset = 1, sensor = 1, light of highway = 1, light of farm road = 4
TIME = 60001, reset = 1, sensor = 1, light of highway = 2, light of farm road = 4
TIME = 72211, reset = 1, sensor = 1, light of highway = 4, light of farm road = 1
TIME = 122011, reset = 1, sensor = 1, light of highway = 4, light of farm road = 2
TIME = 136951, reset = 1, sensor = 1, light of highway = 1, light of farm road = 4
TIME = 136953, reset = 1, sensor = 1, light of highway = 2, light of farm road = 4
```

```
TIME = 136955, reset = 1, sensor = 1, light of highway = 4, light of farm road = 1
TIME = 180000, reset = 1, sensor = 0, light of highway = 4, light of farm road = 1
TIME = 186751, reset = 1, sensor = 0, light of highway = 4, light of farm road = 2
TIME = 201691, reset = 1, sensor = 0, light of highway = 1, light of farm road = 4
TIME = 300000, reset = 1, sensor = 1, light of highway = 1, light of farm road = 4
TIME = 300001, reset = 1, sensor = 1, light of highway = 2, light of farm road = 4
TIME = 311251, reset = 1, sensor = 1, light of highway = 4, light of farm road = 1
TIME = 361051, reset = 1, sensor = 1, light of highway = 4, light of farm road = 2
TIME = 375991, reset = 1, sensor = 1, light of highway = 1, light of farm road = 4
TIME = 375993, reset = 1, sensor = 1, light of highway = 2, light of farm road = 4
TIME = 375995, reset = 1, sensor = 1, light of highway = 4, light of farm road = 1
```

Simulation Waveform for the traffic light controller in Verilog



VHDL code for Traffic light controller on FPGA
What is FPGA Programming? FPGA vs Software programming
Recommended and affordable Xilinx FPGA boards for students
Recommended and affordable Altera FPGA boards for students

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA

33. How to generate a clock enable signal in Verilog
34. Verilog code for PWM Generator
35. Verilog coding vs Software Programming
36. Verilog code for Moore FSM Sequence Detector
37. Verilog code for 7-segment display controller on Basys 3 FPGA

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.